

G4_Test1 — STM32G431KB Bare-Metal Standalone (sin CubeMX)

Proyecto bare-metal para el **STM32G431KBTx** (Cortex-M4) basado en CMSIS-Toolbox (`*.csolution.yml` / `*.cproject.yml`), **sin** depender de STM32CubeMX ni del HAL de ST. El startup y la inicialización de reloj/RAM se controlan directamente a través de dos archivos CMSIS estándar incluidos como código fuente propio del proyecto.

Por qué no se usa el componente `Device:Startup`

El pack `Keil::STM32G4xx_DFP` define el componente `Device:Startup` como variante `STM32CubeMX` : sus archivos reales (`startup_*.s` , `system_stm32g4xx.c`) sólo se completan cuando existe un `.gpdsc` generado por CubeMX. Como este proyecto **no** usa CubeMX, ese componente no entrega ningún archivo (compruébalo: `G4_Test1/RTE/Device/STM32G431KBTx/` sólo tenía el `.sct` plantilla y el header de regiones, nunca el `startup/system`).

La solución: se copiaron las plantillas CMSIS "puras" (no las de CubeMX) directamente desde el pack y se agregaron como archivos explícitos del proyecto, para tener control total y edición libre sobre ellas.

Estructura relevante

```
G4_Test1.csolution.yml      # Solución: packs, target-type (device), build-types
G4_Test1/
├─ G4_Test1.cproject.yml    # Proyecto: componentes, grupos de archivos, linker
├─ main.c
├─ Device/
│   └─ system_stm32g4xx.c   # Copia editable, desde CMSIS Device Templates (no CubeMX)
│   └─ startup_stm32g431xx.s # Copia editable, sintaxis armasm (AC6)
└─ G4_Test1.sct            # Linker script (scatter file) escrito a mano
```

Origen de las plantillas (pack `Keil::STM32G4xx_DFP@1.4.0` , no generadas por CubeMX):

```
Drivers/CMSIS/Device/ST/STM32G4xx/Source/Templates/system_stm32g4xx.c
Drivers/CMSIS/Device/ST/STM32G4xx/Source/Templates/arm/startup_stm32g431xx.s
```

Cambios hechos en `cproject.yml`

- Se **quitó** `component: Device:Startup` .
- Se **mantuvo** `component: ARM::CMSIS:CORE` (headers CMSIS core).
- Se agregó el grupo `Device` con `system_stm32g4xx.c` y `startup_stm32g431xx.s` .
- Se agregó una sección `linker:` apuntando al `.sct` propio:

```
linker:
- script: ./Device/G4_Test1.sct
  for-compiler: AC6
```

`csolution.yml` no requirió cambios: `device: STM32G431KBTx` y el pack `Keil::STM32G4xx_DFP` se mantienen, ya que de ahí siguen viniendo los headers del dispositivo (`stm32g431xx.h`) y el `#define STM32G431xx` , que se activan automáticamente por la selección del device — independientemente de qué componente esté seleccionado.

Memoria (STM32G431KB)

Región	Base	Tamaño
Flash	0x08000000	128 KB
SRAM	0x20000000	32 KB
Stack	—	1 KB
Heap	—	3 KB

Definida directamente (valores literales) en `Device/G4_Test1.sct` .

Compilar

Requiere el compilador **Arm Compiler 6 (AC6)** y CMSIS-Toolbox (`csolution` / `cbuild`).

```
csolution convert G4_Test1.csolution.yml          # valida y genera *.cbuild.yml
cbuild G4_Test1.csolution.yml -c G4_Test1.Debug+STM32G431KBTx
cbuild G4_Test1.csolution.yml -c G4_Test1.Release+STM32G431KBTx
```

Salida en `out/G4_Test1/STM32G431KBTx/<Debug|Release>/` : `G4_Test1.axf` , `G4_Test1.hex` , `G4_Test1.axf.map` .

Verificación realizada

Ambas configuraciones (`Debug` y `Release`) se compilaron y enlazaron exitosamente con AC6 6.24.0:

```
Building context: "G4_Test1.Debug+STM32G431KBTx"
[1/4] Building ASM object ../startup_stm32g431xx.o
[2/4] Building C object  ../main.o
[3/4] Building C object  ../system_stm32g4xx.o
[4/4] Linking C executable ../G4_Test1.axf
Program Size: Code=336 RO-data=488 RW-data=0 ZI-data=5632
Build summary: 1 succeeded, 0 failed
```

En el `.map` resultante se confirmó que la tabla de vectores (sección `RESET` , generada por `startup_stm32g431xx.o`) queda ubicada exactamente en `0x08000000` , inicio de la Flash — es decir, el startup y el linker script propios funcionan correctamente sin ningún archivo generado por CubeMX ni por el componente `Device:Startup` del pack.

Debug / Flash

El destino de depuración configurado en `csolution.yml` sigue siendo ST-Link vía Arm Debugger (`target-types.debugger`), sin cambios respecto al setup original.